

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 12
SOLID PRINCIPLE



Disusun Oleh:
Ahmad Haikal Harahap 105224002

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Pada tugas ini dilakukan analisis serta perancangan ulang terhadap sistem akademik kampus (SIKAD) yang mengalami berbagai permasalahan pada aspek arsitektur perangkat lunak. Sistem yang digunakan sebelumnya memiliki sebuah kelas utama bernama SistemKRSManager yang menangani banyak tanggung jawab sekaligus, mulai dari validasi KRS, perhitungan UKT, pembuatan dokumen PDF, hingga proses penyimpanan data ke database. Kondisi tersebut menyebabkan tingginya ketergantungan antar fungsi sehingga perubahan kecil pada suatu fitur berpotensi menimbulkan gangguan pada fitur lainnya.

Selain itu, ditemukan beberapa pelanggaran terhadap prinsip-prinsip SOLID. Mekanisme perhitungan UKT masih menggunakan banyak percabangan *if-else* yang menyulitkan pengembangan fitur baru. Antarmuka mata kuliah juga dirancang terlalu umum sehingga memaksa setiap jenis mata kuliah mengimplementasikan metode yang sebenarnya tidak dibutuhkan. Di sisi lain, sistem memiliki ketergantungan langsung terhadap database MySQL, sehingga proses migrasi ke teknologi database lain menjadi sulit dilakukan.

Untuk mengatasi berbagai permasalahan tersebut, dilakukan proses *refactoring* dengan menerapkan lima prinsip SOLID, yaitu Single Responsibility Principle (SRP), Open Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), dan Dependency Inversion Principle (DIP). Penerapan prinsip-prinsip tersebut menghasilkan struktur sistem yang lebih modular, fleksibel, mudah dipelihara, serta lebih siap dalam menghadapi kebutuhan pengembangan di masa mendatang, termasuk migrasi ke database berbasis Cloud NoSQL dan penambahan skema perhitungan UKT baru seperti program MBKM.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	nama	String	Menyimpan nama mata kuliah.
2	sks	int	Menyimpan jumlah SKS mata kuliah.
3	database	DatabaseConnection	Menyimpan objek koneksi database yang digunakan sistem.
4	validator	krsValidator	Digunakan untuk melakukan validasi data KRS mahasiswa.
5	repository	krsRepository	Digunakan untuk menyimpan data KRS ke database.
6	printer	krsPdfPrinter	Digunakan untuk mencetak atau menghasilkan dokumen PDF KRS.

7	strategi	StrategiKalkulasiUKT	Menyimpan strategi perhitungan UKT yang digunakan.
8	db	DatabaseConnection	Menentukan jenis database yang digunakan (MySQL atau Cloud NoSQL).
9	mk1	MataKuliah	Menyimpan objek mata kuliah teori.
10	mk2	MataKuliah	Menyimpan objek mata kuliah praktikum.
11	mk3	MataKuliah	Menyimpan objek mata kuliah KKN.
12	totalSKS	int	Menyimpan jumlah total SKS yang diambil mahasiswa.
13	ukt	double	Menyimpan hasil perhitungan UKT mahasiswa.
14	data	String	Menyimpan data KRS yang akan disimpan ke database.

III. Constructor dan Method

No	Nama Method	Jenis Method	Fungsi
1	MataKuliah(String nama, int sks)	Constructor	Membuat objek mata kuliah dengan nama dan jumlah SKS tertentu.
2	getNama()	Getter	Mengambil nilai nama mata kuliah.
3	getSks()	Getter	Mengambil nilai jumlah SKS mata kuliah.
4	tampilkanInfo()	Abstract Method	Menampilkan informasi mata kuliah sesuai jenisnya.
5	MataKuliahTeori(String nama, int sks)	Constructor	Membuat objek mata kuliah teori.
6	tampilkanInfo()	Override Method	Menampilkan informasi mata kuliah teori.

7	MataKuliahPraktikum(String nama, int sks)	Constructor	Membuat objek mata kuliah praktikum.
8	tampilkanInfo()	Override Method	Menampilkan informasi mata kuliah praktikum.
9	alokasiAsistenLab()	Interface Method	Mengalokasikan asisten laboratorium untuk praktikum.

10	cekPeralatanPraktikum()	Interface Method	Memeriksa peralatan yang digunakan dalam praktikum.
11	MataKuliahKKN(String nama, int sks)	Constructor	Membuat objek mata kuliah KKN.
12	tampilkanInfo()	Override Method	Menampilkan informasi mata kuliah KKN.
13	hitungUKT()	Interface Method	Menghitung besaran UKT sesuai jalur mahasiswa.
14	hitungUKT() pada uktReguler	Override Method	Menghitung UKT mahasiswa jalur reguler.
15	hitungUKT() pada uktKaryawan	Override Method	Menghitung UKT mahasiswa jalur karyawan.
16	hitungUKT() pada uktInternasional	Override Method	Menghitung UKT mahasiswa jalur internasional.
17	hitungUKT() pada uktBidikmisi	Override Method	Menghitung UKT mahasiswa jalur bidikmisi.
18	hitungUKT() pada uktMbkm	Override Method	Menghitung UKT mahasiswa program MBKM.

19	saveKRS(String data)	Interface Method	Menyimpan data KRS ke media penyimpanan.
20	saveKRS(String data) pada MySQLDatabaseConnection	Override Method	Menyimpan data KRS ke database MySQL.
21	saveKRS(String data) pada CloudNoSQLConnection	Override Method	Menyimpan data KRS ke database Cloud NoSQL.
22	validasiSKS(int totalSKS)	Procedural	Memvalidasi jumlah SKS yang diambil mahasiswa.
23	cetakPDF()	Procedural	Membuat dan mencetak dokumen PDF KRS.
24	krsRepository(DatabaseConnection database)	Constructor	Menginisialisasi objek repository dengan koneksi database tertentu.
25	simpan(String data)	Procedural	Menyimpan data KRS melalui repository.
26	krsService(krsValidator validator, krsRepository repository, krsPdfPrinter printer, StrategiKalkulasiUKT strategi)	Constructor	Menginisialisasi seluruh komponen layanan KRS.
27	prosesKRS(int totalSKS)	Procedural	Menjalankan proses validasi, perhitungan UKT, penyimpanan data, dan pencetakan PDF.
28	main(String[] args)	Static Method	Menjalankan program utama dan menguji seluruh fitur sistem.

IV. Dokumentasi dan Pembahasan Code 1.

```

public class NoSQLDatabase implements Database {

    @Override
    public void connect() {
        System.out.println("Connected to NoSQL");
    }

    @Override
    public void save(KRS krs) {
        System.out.println("KRS disimpan ke NoSQL");
    }

}

```

Penjelasan:

Kelas NoSQLDatabase merupakan implementasi lain dari interface Database. Kelas ini dibuat untuk mendukung kebutuhan migrasi sistem ke database berbasis Cloud NoSQL. Karena menggunakan interface yang sama dengan MySQLDatabase, proses pergantian database dapat dilakukan tanpa mengubah kode pada kelas lain. Hal ini menunjukkan penerapan prinsip Open Closed Principle (OCP) dan Dependency Inversion Principle (DIP)

2.

```

public interface Database {
    void connect();
    void save(KRS krs);
}

```

Penjelasan:

Kode Database berfungsi sebagai abstraksi untuk proses koneksi dan penyimpanan data. Interface ini dibuat agar sistem tidak bergantung langsung pada jenis database tertentu. Dengan adanya interface ini, kelas lain hanya perlu mengetahui bahwa terdapat operasi connect() dan save(), tanpa harus memahami bagaimana implementasinya. Penerapan ini merupakan contoh Dependency Inversion Principle (DIP) karena modul tingkat tinggi bergantung pada abstraksi, bukan pada implementasi konkret..

3.

```

1  public class MySQLDatabase implements Database {
2
3      @Override
4      public void connect() {
5          System.out.println("Connected to MySQL");
6      }
7
8      @Override
9      public void save(KRS krs) {
10         System.out.println("KRS disimpan ke MySQL");
11     }
12 }

```

Penjelasan:

Kelas MySQLDatabase merupakan implementasi dari interface Database yang bertugas menangani penyimpanan data menggunakan database MySQL. Method connect() digunakan untuk membuat koneksi ke database, sedangkan method save() digunakan untuk menyimpan data. Dengan memisahkan implementasi ini ke dalam kelas tersendiri, perubahan pada teknologi database tidak akan memengaruhi bagian sistem lainnya.

4.

```

public class KRSRepository {

    private Database database;

    public KRSRepository(Database database) {
        this.database = database;
    }

    public void simpan(KRS krs) {
        database.save(krs);
    }

}

```

Penjelasan:

Kelas KRSRepository bertanggung jawab khusus untuk proses penyimpanan data KRS. Kelas ini menerima objek Database melalui constructor sehingga dapat bekerja dengan berbagai jenis database tanpa bergantung pada implementasi tertentu. Pemisahan fungsi penyimpanan data ke dalam kelas khusus merupakan penerapan Single Responsibility Principle (SRP) karena kelas ini hanya memiliki satu tanggung jawab, yaitu mengelola penyimpanan data KRS.

5.

```
public interface UKTStrategy {  
    double hitungUKT(int sks);  
}
```

Penjelasan:

Interface UKTStrategy digunakan untuk mendefinisikan aturan umum perhitungan UKT. Interface ini memiliki method hitungUKT() yang akan diimplementasikan oleh berbagai jenis jalur mahasiswa. Dengan pendekatan ini, setiap algoritma perhitungan UKT dapat dibuat dalam kelas yang berbeda tanpa mengubah kode utama sistem. Penerapan ini mendukung Open Closed Principle (OCP)

6.

```
public class RegulerStrategy implements UKTStrategy {  
  
    @Override  
    public double hitungUKT(int sks) {  
        return sks * 150000;  
    }  
}
```

Penjelasan:

Kelas RegulerStrategy merupakan implementasi dari interface UKTStrategy yang digunakan untuk menghitung UKT mahasiswa jalur reguler. Kelas ini menyimpan rumus perhitungan khusus untuk mahasiswa reguler. Dengan memisahkan rumus ke dalam kelas tersendiri, perubahan aturan UKT jalur reguler tidak akan memengaruhi jalur lainnya.

7.

```
public class KaryawanStrategy implements UKTStrategy {  
  
    @Override  
    public double hitungUKT(int sks) {  
        return sks * 200000;  
    }  
}
```

Penjelasan:

Kelas KaryawanStrategy digunakan untuk menghitung UKT mahasiswa jalur karyawan. Kelas ini memiliki algoritma perhitungan yang berbeda dengan jalur reguler. Penggunaan strategi terpisah membuat sistem lebih fleksibel dan menghindari penggunaan percabangan if-else yang berlebihan pada satu kelas besar.

8.

```
public class MBKMStrategy implements UKTStrategy {  
  
    @Override  
    public double hitungUKT(int sks) {  
        return sks * 100000;  
    }  
}
```

Penjelasan:

Kelas MBKMStrategy dibuat untuk mendukung program Merdeka Belajar Kampus Merdeka (MBKM). Implementasi ini menunjukkan bahwa sistem dapat ditambah dengan skema perhitungan baru tanpa harus memodifikasi kode yang sudah ada. Cukup membuat kelas baru yang mengimplementasikan UKTStrategy, sehingga prinsip Open Closed Principle (OCP) dapat terpenuhi.

9.

```
public class UKTCalculator {  
  
    private UKTStrategy strategy;  
  
    public UKTCalculator(UKTStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public double hitung(int sks) {  
        return strategy.hitungUKT(sks);  
    }  
}
```

Kelas UKTCalculator berfungsi sebagai penghubung antara sistem dengan berbagai strategi perhitungan UKT. Kelas ini menerima objek UKTStrategy melalui constructor dan menggunakan strategi tersebut saat melakukan perhitungan. Dengan demikian, kelas ini tidak perlu mengetahui detail rumus yang digunakan. Pendekatan ini menggunakan pola desain Strategy Pattern dan mendukung fleksibilitas sistem.

10.

```
public interface MataKuliah {  
    String getNama();  
}
```

Penjelasan:

Interface MataKuliah digunakan untuk mendefinisikan perilaku dasar yang dimiliki oleh seluruh jenis mata kuliah. Interface ini hanya berisi method getNama() sehingga semua jenis mata kuliah dapat mengimplementasikannya tanpa dipaksa

memiliki fitur yang tidak diperlukan. Perancangan ini merupakan penerapan **Interface Segregation Principle (ISP)**.

11.

```
public interface Praktikum {  
    void alokasiAsistenLab();  
    void cekPeralatan();  
}
```

Penjelasan:

Interface Praktikum dibuat khusus untuk mata kuliah yang memiliki kegiatan praktikum. Interface ini menyediakan method `alokasiAsistenLab()` dan `cekPeralatan()`. Dengan memisahkan fungsi praktikum ke interface tersendiri, mata kuliah teori tidak perlu mengimplementasikan method yang tidak digunakan.

12.

```
public class MataKuliahTeori implements MataKuliah {  
  
    @Override  
    public String getNama() {  
        return "Algoritma dan Pemrograman";  
    }  
}
```

Penjelasan:

Kelas `MataKuliahTeori` merupakan implementasi dari interface `MataKuliah`. Karena mata kuliah teori tidak memerlukan laboratorium maupun peralatan praktikum, kelas ini hanya mengimplementasikan method yang benar-benar dibutuhkan. Pendekatan ini menghindari penggunaan `UnsupportedOperationException` yang sebelumnya terjadi pada sistem lama dan memenuhi prinsip **Liskov Substitution Principle (LSP)** serta **Interface Segregation Principle (ISP)**.

13.

```

public class MataKuliahPraktikum
{
    implements MataKuliah, Praktikum {

        @Override
        public String getNama() {
            return "Praktikum PBO";
        }

        @Override
        public void alokasiAsistenLab() {
            System.out.println("Asisten dialokasikan");
        }

        @Override
        public void cekPeralatan() {
            System.out.println("Peralatan dicek");
        }
    }
}

```

Penjelasan:

Kelas MataKuliahPraktikum mengimplementasikan interface MataKuliah dan Praktikum. Selain memiliki nama mata kuliah, kelas ini juga menyediakan fungsi alokasi asisten laboratorium dan pengecekan peralatan praktikum. Pemisahan ini membuat setiap jenis mata kuliah hanya memiliki perilaku yang sesuai dengan kebutuhannya.

14.

```

public class KRSValidator {

    public boolean validasiPrasyarat() {
        return true;
    }

}

```

Penjelasan:

Kelas KRSValidator bertanggung jawab untuk melakukan validasi prasyarat mata kuliah sebelum mahasiswa melakukan pengisian KRS. Kelas ini dibuat terpisah agar proses validasi tidak bercampur dengan fungsi lain seperti penyimpanan data atau perhitungan UKT. Hal tersebut merupakan penerapan Single Responsibility Principle (SRP).

15.

```
public class PDFGenerator {
    public void generateKRS() {
        System.out.println("PDF berhasil dibuat");
    }
}
```

Penjelasan:

Kelas PDFGenerator memiliki tanggung jawab khusus untuk membuat dokumen KRS dalam format PDF. Dengan memisahkan fungsi pembuatan dokumen ke kelas tersendiri, perubahan pada tampilan atau format PDF tidak akan memengaruhi proses penyimpanan data maupun perhitungan UKT. Ini merupakan contoh penerapan Single Responsibility Principle (SRP).

16.

```
public class Main {
    public static void main(String[] args) {
        UKTCalculator calculator =
            new UKTCalculator(new MBKMStrategy());
        System.out.println(
            "UKT = " + calculator.hitung(20)
        );
        Database db = new NoSQLDatabase();
        KRSRepository repo =
            new KRSRepository(db);
        repo.simpan(new KRS());
    }
}
```

Penjelasan:

Kelas Main digunakan sebagai titik awal eksekusi program dan berfungsi untuk menguji seluruh komponen yang telah dibuat. Pada kelas ini dilakukan pembuatan objek strategi UKT, pemilihan jenis database, serta pemanggilan repository untuk menyimpan data. Kelas ini menunjukkan bagaimana seluruh komponen yang telah dirancang dapat bekerja sama secara fleksibel melalui penerapan prinsip-prinsip SOLID.

V. Kesimpulan

Berdasarkan hasil analisis yang telah dilakukan, dapat disimpulkan bahwa sistem SIAKAD lama memiliki banyak permasalahan karena tidak menerapkan prinsip-prinsip SOLID dengan baik. Hal ini menyebabkan kode menjadi sulit dikembangkan, sulit dipelihara, serta rentan mengalami error ketika terjadi perubahan pada sistem.

Untuk mengatasi masalah tersebut, dilakukan perancangan ulang dengan menerapkan prinsip **Single Responsibility Principle (SRP)**, **Open Closed Principle (OCP)**, **Liskov Substitution Principle (LSP)**, **Interface Segregation Principle (ISP)**, dan **Dependency Inversion Principle (DIP)**. Dengan penerapan prinsip-prinsip tersebut, setiap kelas memiliki tugas yang lebih jelas, proses perhitungan UKT menjadi lebih fleksibel, penggunaan interface menjadi lebih tepat, serta sistem tidak lagi bergantung pada satu jenis database tertentu.

Hasilnya, sistem menjadi lebih modular, mudah dipahami, mudah dikembangkan, dan siap menghadapi kebutuhan di masa depan seperti penambahan jalur mahasiswa baru (MBKM) maupun migrasi database dari MySQL ke Cloud NoSQL tanpa harus mengubah banyak kode yang sudah ada. Dengan demikian, penerapan SOLID dapat meningkatkan kualitas dan maintainability perangkat lunak secara keseluruhan.

VI. Daftar Pustaka

Oracle. (2023). *Java Documentation*. <https://docs.oracle.com/en/java/>